COS30018 – Intelligent Systems

Task C.2 – Data processing 1

I created a new function called load_and_process_data with the following attributes:

```
def load_and_process_data(
    company,
    start_date,
    end_date,
    feature_columns=["Open", "High", "Low", "Close", "Volume"],
    nan_strategy="drop",
    split_method="ratio",
    train_ratio=0.8,
    split_date=None,
    scale_features=True,
    save_local=True,
    local_dir="data",
    force_download=False,
    prediction_days=60,
):
```

This is what I have implemented for the task of 1.d which requires the function to allow you to store the downloaded data on your local machine for future uses and to load the data locally to save time, and I also coded a part to filter the columns if possible. Now the file name will include the ticker as well as the date range and end with ".csv" so that every query in the future will never read the wrong dataset file, and if the file is already existed in the disk drive, it will ship the downloading part. And the parse_dates is set to "True" so that when it read the csv file, the index column will become DatetimeIndex instead of string, which is needed for the split using date part in later.

```
#-----------------------------------------------------------------------------
# Load or Download Data
#-----------------------------------------------------------------------------
    os.makedirs(local_dir, exist_ok=True)
    cache_path = os.path.join(local_dir, f"{company}_{start_date}_{end_date}.csv")

    if not force_download and os.path.exists(cache_path):
        df = pd.read_csv(cache_path, index_col=0, parse_dates=True)
    else:
        df = yf.download(company, start=start_date, end=end_date)
        if isinstance(df.columns, pd.MultiIndex):
            df.columns = df.columns.get_level_values(0)
        if save_local:
            df.to_csv(cache_path)

#-----------------------------------------------------------------------------
# Filter feature columns
#-----------------------------------------------------------------------------
    feature_columns = [col for col in feature_columns if col in df.columns]
    df = df[feature_columns].copy()
```

This is what I have implemented to handle the Not a Number (NaN) issue in the data. It will have 4 options with the first one is “drop”, which means that it will delete the data marked as “Nan”. The second one is “fill_ff” or “forward fill”, which will replace the NaN value with the value beffore it, after that it will still drop the data cell in case if the first data cell is NaN and there is no value before it to fill in. “fill_ bf” or “backward fill” is the same as “fill_ff” but opposite, meaning that it will fill the value NaN with the value of the data cell after it. Finally, the “fill_mean” will take the average value of the whole data column and fill it in the cell with the NaN value.

```
#-----------------------------------------------------------------------------
# Handle NaN values
#-----------------------------------------------------------------------------
    if nan_strategy == "drop":
        df.dropna(inplace=True)
    elif nan_strategy == "fill_ff":
        df.ffill(inplace=True)
        df.dropna(inplace=True)
    elif nan_strategy == "fill_bf":
        df.bfill(inplace=True)
        df.dropna(inplace=True)
    elif nan_strategy == "fill_mean":
        df.fillna(df.mean(), inplace=True)
```

This is what I have implemented to split the data into train/test data. There are 3 ways to split the data with the first one is to split it using ratio. For example, if there are 1000 lines of data in the dataset and we set the ratio to 0.8, the lines from 0 to 799 will be use as train data, and the rest will be use as test data with the order of time kept intact. The second method to split the data is split by date, it will use “pd.Timestamp” to compare with the “DatetimeIndex”, every line that is before the “split_date” will be use as train data, and lines that are after the “split_date” will be use as test data. The final method to split the data is to split it randomly, it will randomly take a set ratio of data without following the order of time to use as train data, and the rest will be use as test data. The random_state=42” is there to make sure that the algorithm will only randomize the data on a set state, unless you change the ratio, the outcome of the randomized split will always the same.

```
#-----------------------------------------------------------------------------
# Split into train / test
#-----------------------------------------------------------------------------
    if split_method == "ratio":
        split_idx = int(len(df) * train_ratio)
        train_df = df.iloc[:split_idx]
        test_df  = df.iloc[split_idx:]
    elif split_method == "date":
        train_df = df[df.index < pd.Timestamp(split_date)]
        test_df  = df[df.index >= pd.Timestamp(split_date)]
    elif split_method == "random":
        train_df = df.sample(frac=train_ratio, random_state=42)
        test_df  = df.drop(train_df.index)
```

This is what I have implemented for the scaling function. Each column need to have a separated range because there are a few type of column, the “Close” feature can be in range of 100 while the “Volume” feature can be a million. It’s important to only “fit_transform” on the train data, and only “transform” on the test data so that it won’t cause data leakage. All the scalers will be saved into the dict “scalers” with the key is the name of the column feature so that it is easier to convert them back to their real values.

```
#-----------------------------------------------------------------------------
# Scale the features
#-----------------------------------------------------------------------------
    scalers = {}
    train_scaled = train_df.copy().astype(float)
    test_scaled  = test_df.copy().astype(float)

    if scale_features:
        for col in feature_columns:
            scaler = MinMaxScaler(feature_range=(0, 1))
            train_scaled[col] = scaler.fit_transform(
                train_df[col].values.reshape(-1, 1)
            ).flatten()
            test_scaled[col] = scaler.transform(
                test_df[col].values.reshape(-1, 1)
            ).flatten()
            scalers[col] = scaler
```